SMART EV CHARGING STATION PLANNING AND ENERGY ALLOCATION SYSTEM

Project Description

Course: CT-363 — Design and Analysis of Algorithms

Project focus: Greedy Minimum Spanning Forest + Exact Multi-Knapsack Dynamic Programming

Application context: Urban Pakistan

Prepared as a project description and implementation specification

Abstract

This project proposes a Smart EV Charging Station Planning and Energy Allocation System for an urban electric-vehicle charging network. The system addresses two linked planning problems using two algorithmic paradigms required for the course: (1) a Greedy algorithm, Kruskal's algorithm with a Disjoint Set Union (DSU) structure, to construct a minimum-cost connection network over candidate charging-station sites; and (2) exact Dynamic Programming (DP) for a multi-knapsack allocation problem that assigns each EV charging request to at most one eligible active station while respecting station energy capacities and maximizing total service benefit. The network algorithm must handle unavailable roads, disconnected components, deterministic tie-breaking, and an incremental update after three roads become unavailable. The allocation algorithm must handle capacity limits, requests that cannot fit any station, unserved demand, assignment reconstruction, and comparison with brute force on small instances. The evaluation will use a named dataset containing at least 10 network locations, at least 20 road connections, and at least 20 EV requests. Sensitivity experiments will compare baseline results with a scenario in which the two highest-severity locations are removed. The project will report correctness arguments, exact complexity derivations, reproducible outputs, edge-case tests, limitations, and a clear distinction between modeled outcomes and assumptions.

1. Introduction and Background

Electric-vehicle adoption creates a need to plan charging infrastructure and allocate limited energy fairly and efficiently. In an urban setting, a plan must consider the cost of connecting charging sites, the availability of road links, limited station capacity, and competing charging requests. These requirements can conflict: a low-cost network may not provide the widest coverage; a station with spare capacity may be inaccessible to a request; and serving a high-value request may leave less energy for other users.

This project applies algorithm design rather than claiming to build a production-ready city deployment tool. Candidate locations, road costs, station capacities, request energy requirements, and service benefits will be represented in a transparent test dataset. Results will be interpreted within those assumptions. A minimum spanning tree (MST) minimizes the total cost of connecting all vertices when the graph is connected; when it is disconnected, the correct output is a minimum spanning forest (MSF), one minimum-cost tree per connected component.

The project does not solve the geographic facility-location problem of deciding where new stations should be built. It assumes that candidate sites have already been identified. Kruskal's algorithm chooses a low-cost set of connections among those sites; DP allocates available energy among eligible requests at the active stations.

2. Problem Statement

Given a graph of candidate charging-station locations and road links, some of which may be unavailable, construct a minimum-cost connection network or minimum spanning forest. Then, given a set of EV charging requests and a limited number of active stations with finite energy capacities, assign each request to at most one eligible station or leave it unserved. The assignment must not exceed any station's capacity and must maximize total defined service benefit. The system must explain how network connectivity affects assignment eligibility, detect infeasible requests, support a targeted network update after three road failures, and provide reproducible experimental results.

2.1 Research Questions

- How can a Greedy MST/MSF algorithm connect the available candidate sites at minimum total modeled cost?
- How should the system respond when the graph is disconnected or selected road links become unavailable?
- How can an exact multi-knapsack DP assign requests to stations without assigning a request more than once or exceeding energy capacity?
- How do station capacities, road failures, and removal of high-severity locations affect network cost, components, requests served, energy allocated, and total benefit?

- How do the proposed algorithms compare with full recomputation and brute-force solutions on appropriately sized test cases?

3. Objectives

- Model candidate charging sites and available road connections as a weighted, undirected graph.
- Implement Kruskal's algorithm using DSU with path compression and union by rank/size.
- Exclude unavailable roads before processing and return a minimum spanning forest when the graph is disconnected.
- Use a deterministic equal-cost tie-break based on road cost, then the severity priority of the endpoints, then stable endpoint IDs.
- Implement an incremental update after three roads become unavailable and compare its result and cost with a full Kruskal recomputation.
- Formulate and implement exact multi-knapsack DP for assigning each request to at most one active station.
- Reconstruct the selected request-to-station assignments and report unserved or infeasible requests explicitly.
- Evaluate correctness, time and space complexity, edge cases, sensitivity, and scalability using reproducible data.

4. Scope, Assumptions, and Definitions

Item	Definition used in this project
Candidate site / vertex	A named candidate charging-station location in the network graph.
Road / edge	An undirected connection between two candidate sites with a non-negative modeled construction or connection cost.
Unavailable road	An edge marked closed, submerged, damaged, or otherwise unavailable; it is excluded from the active graph.
Location severity	A predefined integer priority score used for equal-cost tie-breaking and the required sensitivity scenario. It is not automatically a safety guarantee.
Active station	A station selected for energy allocation and given a finite energy capacity. The network may contain more candidate sites than active allocation stations.
EV request	A charging request with a unique ID, energy requirement, service benefit, and mapped origin/access node.
Request eligibility	A request may be assigned only to an active station in the same connected component as its mapped origin/access node. If the dataset does not model access nodes explicitly, the limitation must be stated and no reachability claim may be made.
Energy unit	Capacities and request energy are converted to integer energy units for DP. The chosen unit size and rounding policy must be documented.
Benefit	A non-negative numeric score representing the modeled value of serving a request. Any use of urgency or time-decay must be expressed in a stated formula.
Unserved request	A request that is not assigned because of capacity, eligibility, or an explicit decision to leave it unserved.

Assumptions: road costs and station capacities are known inputs; all edge costs are non-negative; the graph is undirected; each request can be served at most once; each request consumes its full specified energy requirement; and benefit values are fixed for a run. Unless the implementation explicitly models charging duration, queues, power limits, electricity prices, and safety constraints, those factors are discussion points rather than optimized variables. The system will not claim to optimize a factor that is absent from its mathematical model.

5. Conflicting Engineering Requirements

Requirement / conflict	How it is represented	What will be measured / limitation

Connection cost vs coverage	Kruskal minimizes total edge cost over available links; it does not directly maximize geographic coverage.	Total selected cost, number of connected components, and locations in each component.
Availability / road failure	Unavailable roads are removed before MST/MSF computation; three-road failure is a required update experiment.	Before/after forest, component changes, replacement edges, and runtime/cost comparison.
Energy capacity vs demand	Each active station has a capacity constraint in the multi-knapsack DP.	Capacity utilization, requests served/unserved, and energy assigned.
Benefit vs fairness / urgency	The primary objective maximizes the sum of declared benefit values. Urgency may affect benefit only through an explicit documented formula.	Total benefit and, optionally, urgent-request service rate. The model does not guarantee fairness unless a fairness constraint/objective is added.
Time-decay vs computation	If included, request benefit may decrease with waiting time using a stated function, such as b_i(t)=b_i^0 exp(-lambda_i t); otherwise time-decay is outside the optimized model.	State units, parameter values, and compare assignments with and without decay if implemented.
Safety vs low cost	Hazardous/unavailable edges are excluded. Additional safety risk can be modeled as a hard edge filter or separate risk constraint only if data are provided.	Explain that low connection cost alone does not imply a safe route.
Exactness vs scalability	Exact multidimensional DP is used on a bounded number of active stations and integer energy units; larger instances are used to study state growth or a clearly identified sparse-state exact implementation.	Report actual state counts, runtime, memory behavior, and any size limits.

6. System Architecture and Integrated Workflow

The implementation will use a modular pipeline:

1. Input and validation: load named locations, road edges, availability flags, severity scores, active station capacities, and EV requests. Reject duplicate IDs, negative capacities, invalid endpoints, and malformed data.
2. Active graph construction: remove unavailable roads and build the graph from the remaining edges.
3. Initial network optimization: sort edges by the documented tie-break order and run Kruskal with DSU to produce an MST or MSF.
4. Reachability / eligibility: map each EV request to an origin/access node and identify which active stations are in the same connected component. Requests with no eligible station are reported as unserviceable by the modeled network.
5. Energy allocation: run exact multi-knapsack DP over eligible request-station assignments and station capacities. Reconstruct assignments and verify all constraints.
6. Failure update: mark three selected roads unavailable, update the existing forest without simply rerunning the full algorithm as the primary method, and separately run full Kruskal as the comparison baseline.
7. Reporting: output selected edges, total connection cost, components, request assignments, unserved requests, station utilization, total energy, total benefit, test results, and runtime comparisons.

Important integration rule: the network and allocation stages must be connected by the eligibility rule above. If the project only computes an MST and a separate allocation without using graph reachability or any other defined network output, it must describe them as two related but independent analyses rather than claiming the MST causes the allocation result.

7. Algorithm A — Greedy Minimum Spanning Forest

7.1 Model

Let G=(V,E) be an undirected weighted graph. Each edge e=(u,v) has a non-negative cost w(e), and each vertex has a predefined severity score s(v). Each edge also has an availability flag. Let E_active contain only available edges. Sort E_active lexicographically by (cost ascending, combined endpoint severity s(u)+s(v) descending, min(endpoint ID) ascending, max(endpoint ID) ascending). Severity is considered only when edge costs are equal; it does not change the minimum total cost of a valid MST/MSF, though it may select a particular forest among equal-cost alternatives.

7.2 Pseudocode

```
KRUSKAL-MSF(V, E):
    A <- all available edges in E
    sort A by (cost ascending, severity tie-break, stable endpoint IDs)
    initialize DSU with one set per vertex
    F <- empty set
    for each edge (u, v, cost) in A:
        if FIND(u) != FIND(v):
            add (u, v, cost) to F
            UNION(u, v)
    return F and DSU components
```

7.3 Correctness Argument

Greedy-choice property: consider an edge e chosen by Kruskal that joins two different DSU components. The current components define a cut between the two sides. Since e is the lowest-ranked available edge crossing that cut under the fixed sorting order, the cut property implies that e is a safe choice for a minimum spanning tree of the component (with deterministic tie-breaking among equal-cost choices).

Optimal substructure: after adding a safe edge, the remaining task is to connect the resulting components using the remaining edges at minimum additional cost. An optimal forest containing the safe edge can be completed by an optimal solution to the reduced problem; otherwise replacing its remainder with a cheaper completion would improve the original forest, contradicting optimality.

Disconnected graph: if the available graph is disconnected, no algorithm can connect all vertices using only available edges. Kruskal returns a minimum spanning forest. The implementation must report every component and list the location names in each component, including isolated vertices.

7.4 Complexity

Let V be the number of vertices and E the number of input edges. Filtering unavailable edges costs O(E); sorting costs O(E log E); DSU operations across all edges cost O(E α(V)) amortized with path compression and union by rank/size; initialization costs O(V). Total time is O(V + E log E + E α(V)), conventionally simplified to O(E log E) for a nontrivial graph. Space is O(V) for DSU and component metadata plus O(E) to store/sort edges, so total auxiliary/input working storage is O(V+E).

8. Incremental Update After Three Road Failures

The experiment will select three initially available roads and mark them unavailable. The update algorithm must modify the existing forest rather than simply calling Kruskal again and labeling that result incremental.

```
INCREMENTAL-DELETE-EDGES(F, E_active, failed_edges):
    remove failed_edges from E_active
    for each failed edge e=(u,v):
        if e is not currently in F:
            continue                    // forest remains valid
```

```
        remove e from F
        find the two components created by removing e
        C1, C2 <- vertices on the two sides of the cut
        candidates <- surviving available edges (x,y) with
                      x in C1 and y in C2
        if candidates is not empty:
            choose the minimum-ranked candidate under the same
            edge order used by Kruskal
            add that candidate to F
        else:
            retain the split as separate forest components
    return updated F
```

Implementation detail: the component search and candidate scan must use the current graph after all failures processed so far. A simple implementation can scan surviving edges to find the cheapest edge crossing the cut. This is an incremental repair strategy, but it may not always be faster asymptotically than full recomputation; its actual runtime must be measured. For a small graph, the main claim is that it repairs the affected forest locally and is checked against a full recomputation, not that it is universally faster.

Validation: after the update, verify that all selected edges are available, the forest is acyclic, each connected component of the remaining graph is spanned, and the total cost equals the full-Kruskal baseline. If equal-cost alternatives exist, compare total cost and validity; exact edge-set equality is not required unless tie-breaking yields a unique result.

Incremental-update complexity: in this straightforward implementation, finding components after a tree-edge deletion and scanning all surviving edges for the minimum replacement can each take O(V+E) time. With r failed roads, a conservative bound is O(r(V+E)); here r=3, so the update is O(V+E) up to a small constant factor. This is a simple repair algorithm, not a fully dynamic MST data structure. Report measured runtime and compare it with full Kruskal rather than assuming it is faster.

9. Algorithm B — Exact Multi-Knapsack Dynamic Programming

9.1 Inputs and objective

Let there be n EV requests and k active stations. Request i has energy requirement w_i (integer energy units) and benefit b_i. Station j has capacity C_j. A request can be assigned to at most one eligible station, or skipped. Let x_ij be 1 when request i is assigned to station j and 0 otherwise. The model is:

```
Maximize:  sum_i sum_j b_i * x_ij
Subject to:
    sum_j x_ij <= 1                          for every request i
    sum_i w_i * x_ij <= C_j                  for every station j
    x_ij = 0 if request i is ineligible for station j
    x_ij in {0,1}
```

This is a multiple-knapsack allocation problem. It is not correct to solve a separate independent knapsack for each station because that could assign the same request to multiple stations. The DP processes each request once and considers either skipping it or assigning it to exactly one eligible station.

9.2 State and recurrence

Define DP[i,c1,...,ck] as the maximum benefit obtainable using the first i requests when station j has a capacity budget of c_j energy units. These c_j values are capacity limits for the subproblem, not capacities that have already been consumed. The base case is DP[0,c1,...,ck]=0 for every feasible capacity vector. For request i, skipping it preserves the state value. Assigning it to an eligible station j is allowed only if w_i <= c_j:

```
DP[i, c1, ..., ck] = max(
    DP[i-1, c1, ..., ck],                          // skip request i
    b_i + DP[i-1, c1, ..., c_j - w_i, ..., ck]     // assign to j
       for each eligible j with c_j >= w_i
)
```

The final optimum is DP[n,C1,...,Ck], where C_j is the original capacity of station j. Reconstruction follows the recorded decision for each state: if request i was assigned to station j, record that assignment and move to state (i-1, c_1,...,c_j-w_i,...,c_k); if it was skipped, move to (i-1,c_1,...,c_k). This convention is internally consistent with the capacity-budget state definition. A request with w_i greater than every eligible station capacity can never be assigned. It must be reported as too large for any eligible station, not silently dropped. A request that is individually feasible but left unassigned because the total demand exceeds capacity must be listed as unserved due to competition for limited capacity.

A request with w_i greater than every eligible station capacity can never be assigned. It must be reported as too large for any eligible station, not silently dropped. A request that is individually feasible but left unassigned because the total demand exceeds capacity must be listed as unserved due to competition for limited capacity.

9.3 Complexity and feasibility control

For integer capacities C_1,...,C_k, the dense state-space size is S = product over j of (C_j+1). Each of n requests considers up to k assignment transitions plus skipping. Time is O(n k S). Storing all DP layers for straightforward reconstruction uses O(n S) space; storing only two value layers uses O(S) space but requires a separate reconstruction strategy or retained decisions. These bounds are pseudo-polynomial because they depend on numeric capacity values, not only the bit-length of those values.

Critical feasibility constraint: a dense DP over 10 stations with capacity 100 units each has 101^10 states, which is computationally infeasible. Therefore, the exact-DP evaluation will use a deliberately bounded number of active allocation stations (recommended 3–4) and documented integer energy units, while the network graph can still contain at least 10 candidate locations. The report must state the actual k and capacities used. If a sparse-state exact DP is implemented for more stations, report its observed state count and retain the dense worst-case complexity warning; sparse storage does not remove worst-case exponential growth in k.

9.4 Pseudocode

```
MULTI-KNAPSACK-DP(requests, stations, eligibility):
    convert all energies/capacities to documented integer units
    initialize DP[0, capacity_vector] = 0
    for i from 1 to n:
        for each feasible remaining-capacity vector c:
            DP[i,c] <- DP[i-1,c]                // skip request i
            decision[i,c] <- SKIP
            for each eligible station j:
                if request[i].weight <= c[j]:
                    c2 <- c with c2[j] = c[j] - request[i].weight
                    candidate <- benefit[i] + DP[i-1,c2]
                    if candidate > DP[i,c]:
                        DP[i,c] <- candidate
                        decision[i,c] <- ASSIGN_TO(j)
    backtrack decisions from full capacity vector
    validate: each request assigned at most once;
              each station load <= capacity;
              every assignment is eligible
    return assignments, total benefit, loads, unserved requests
```

Tie-breaking between equal-benefit DP choices must be deterministic (for example, prefer the solution serving more requests, then lower request ID, then lower station ID), or the implementation must document that it retains the first

encountered optimum. If the secondary objective “serve more requests” is used, it must be encoded explicitly in the comparison key and described in the report.

10. Brute-Force Baseline

For small test instances, compare the exact DP result against exhaustive enumeration. Each request has k possible station assignments plus the option to skip, so the number of raw assignments is (k+1)^n. Validating capacity and eligibility for each assignment adds polynomial work, giving a simple upper-bound description such as O((k+1)^n · poly(n,k)). The brute-force implementation is for small n only. It must solve the same objective, use the same eligibility constraints, and agree with DP on optimal benefit; when multiple optimal assignments exist, compare objective values and constraint validity rather than requiring identical assignments.

11. Dataset and Worked Example Requirements

The final implementation must include a concrete, named, reproducible dataset—not only a plan to create one—with at least 10 network locations, at least 20 road connections, and at least 20 EV requests. Each row must include a stable ID and every field used by an algorithm. The dataset may be synthetic, but it must be labeled synthetic and its values must be plausible and consistently used.

Dataset table	Required fields
Locations	location_id, location_name, severity_score, active/removed status, component/access mapping if used.
Road connections	edge_id, endpoint_u, endpoint_v, cost, availability flag, optional risk attribute if modeled.
Active stations	station_id/location_id, energy_capacity_kWh, integer capacity units, active status.
EV requests	request_id, origin/access node, energy requirement_kWh, integer energy units, benefit, eligibility rule inputs, optional urgency/time-decay parameters.

The worked example must show actual computed outputs for the same dataset: (1) the sorted eligible edge order or a concise Kruskal trace; (2) selected MST/MSF edges and total cost; (3) component membership and isolated locations; (4) each EV request assigned to a station or marked unserved, with energy and benefit; (5) station loads and capacity checks; and (6) total assigned energy and total benefit. Illustrative or hand-written output must be labeled illustrative unless generated by the submitted code. The final report should include the exact command or steps needed to reproduce it.

12. Required Experiments and Sensitivity Analysis

12.1 Baseline experiment

- Run Kruskal/MSF on the complete dataset after filtering unavailable roads.
- Run the multi-knapsack DP using the resulting component-based eligibility rules.
- Record selected edges, total connection cost, components, assigned/unserved requests, station utilization, total energy, total benefit, runtime, and memory/state counts where available.

12.2 Three-road failure / incremental update

- Choose three initially available roads and record them before changing the graph.
- Mark those roads unavailable and repair the existing forest using the incremental deletion algorithm.
- Run full Kruskal on the modified graph as a correctness baseline.
- Compare edge validity, component structure, total cost, selected edges, and measured runtime. Explain any tie-driven edge differences.

12.3 Remove the two highest-severity locations

Identify the two locations with the highest severity scores before the experiment. Remove those vertices and every incident road, and deactivate any station at those locations. Rerun both algorithms from the modified input. Report the changes in selected edges, total connection cost, number and membership of components, eligible station set, requests served and unserved, total energy allocated, and total benefit. Explain why a high-severity location's removal may reduce coverage or split a component. If severity is only a tie-break and not part of the objective, do not claim that Kruskal directly maximizes severity coverage.

12.4 Capacity and demand sensitivity

- Reduce one or more station capacities by a documented percentage and rerun DP.
- Increase request demand or add requests while preserving unique IDs, then record how benefit, energy utilization, and unserved requests change.
- Optionally compare the allocation with time-decayed benefits against the fixed-benefit baseline, but only if time-decay is actually implemented.

13. Mandatory Edge Cases and Tests

Test	Input condition	Expected behavior / assertion
Disconnected graph	At least two components, including a possible isolated location.	Return an MSF; list each component and isolated location by name; do not invent unavailable connections.
Over-demand	Total feasible request energy exceeds aggregate station capacity.	Respect all station capacities; leave some requests unserved if necessary; report total benefit and unserved IDs.
Too-heavy request	Request energy exceeds every eligible station capacity.	Do not assign it; report it as individually infeasible and explain why.
Unavailable edge	An edge is marked unavailable.	Edge is never selected by initial or updated forest.
Cycle prevention	Several low-cost edges form a cycle.	DSU rejects an edge that joins vertices already in the same component.
Equal-cost edges	Two or more edges have the same cost.	Tie-break is deterministic and documented; total forest cost remains minimal.
Duplicate assignment guard	A request could appear attractive to multiple stations.	DP assigns it to at most one station.
Zero requests / zero capacity	Empty request set or a station with zero capacity.	Return a valid empty/partial allocation without crashing.
Incremental failure	One failed edge is in the forest and another is not.	Repair only where needed; result agrees in cost and validity with full recomputation.

At least the three assignment-mandated edge cases—disconnected graph, over-demand, and too-heavy item—must be run in the submitted code and shown with actual output. The remaining tests strengthen the evaluation.

14. Evaluation Metrics and Results Presentation

- Network: selected edge list, total cost, number of components, component membership, unavailable edges excluded, and incremental-versus-full recomputation comparison.
- Allocation: requests served/unserved, total benefit, total energy assigned, load/capacity per station, eligibility validity, and reconstruction correctness.
- Efficiency: measured runtime for initial Kruskal, incremental repair, full Kruskal, DP, and brute force on small inputs; state count and memory usage where practical.
- Correctness: forest acyclicity and spanning checks, all edges available, capacity constraints, no duplicate assignments, all assignments eligible, and DP objective matching brute force on small instances.

- Sensitivity: differences from baseline after road failures, capacity changes, demand increases, and removal of the two highest-severity locations.

Runtime comparisons should use the same machine, programming language, and input for each compared method. Repeat runs where feasible and report the measurement method. Do not claim a speedup from one small run without qualification.

15. Implementation Plan and Deliverables

Module	Deliverable
Data model and validation	Input files, schema, validation errors, and named synthetic dataset.
Greedy network algorithm	Kruskal/MSF with DSU, unavailable-edge filtering, stable tie-break, component names, and correctness checks.
Incremental repair	Three-road deletion update, replacement-edge scan, and comparison against full Kruskal.
DP allocator	Exact multi-knapsack DP, assignment reconstruction, eligibility filtering, and explicit infeasibility/unserved reporting.
Baseline verifier	Brute-force solver for small instances and automated agreement checks.
Testing	At least three required edge-case runs with captured output, plus additional tests.
Results and analysis	Baseline, road-failure, high-severity-removal, capacity, and demand experiments.
Report	At least six pages, with required sections, pseudocode, proofs, complexity derivations, results, limitations, and references.

16. Proposed Report Structure

8. Title page
9. Abstract
10. Introduction and background
11. Literature review: MST/Kruskal/DSU and multi-knapsack DP, with credible references
12. Problem formulation, objectives, assumptions, and conflicting requirements
13. System architecture and block diagram
14. Methodology and pseudocode for every algorithm
15. Correctness arguments: greedy-choice property, optimal substructure, and DP recurrence justification
16. Time and space complexity derivations, including DSU and DP state-space analysis
17. Implementation details and dataset description
18. Results: baseline, worked example, edge cases, incremental update, brute-force comparison, and sensitivity experiments
19. Discussion: trade-offs, limitations, scalability, and threats to validity
20. Conclusion and future work
21. References and appendices containing data/code/output where appropriate

17. Limitations and Ethical/Practical Considerations

- Synthetic data cannot establish real-world construction cost, demand, or traffic performance. Any real deployment would require validated local data.
- An MST minimizes total connection-edge cost under the modeled graph; it does not automatically optimize route travel time, charging coverage, resilience, safety, or station placement.
- A fixed benefit score can encode the designer's priorities. The score formula and weights must be disclosed; benefit maximization alone does not guarantee equitable service.

- The DP assumes discrete energy units and fixed request requirements. Rounding can affect feasibility, so the chosen unit must be small enough for the study and its effect must be discussed.
- Exact multidimensional DP grows rapidly with the number of active stations and capacity units. Any restricted station count or state compression must be stated transparently.
- The model does not represent queueing, charger power, variable charging duration, grid limits, prices, traffic dynamics, or detailed safety risk unless those variables are explicitly added.

18. Conclusion

The revised project combines two distinct and assessable algorithmic paradigms: Greedy optimization for a minimum spanning forest and exact Dynamic Programming for multi-station energy allocation. It addresses the core course expectations by requiring formal problem definitions, correctness reasoning, exact complexity analysis, working code, reproducible examples, and edge-case testing. The essential technical safeguards are to exclude unavailable roads, report disconnected components by name, implement and validate the three-road incremental update, ensure each request is assigned at most once, reconstruct DP decisions, control the multidimensional state-space size, and connect network connectivity to allocation eligibility. The final submission must replace planned or illustrative results with outputs produced by the actual implementation.

References to Consult and Cite in the Final Report

Use and cite the course materials supplied by the instructor, plus standard authoritative algorithm references that the team has actually consulted. Suitable starting points include:

- Cormen, T. H., Leiserson, C. E., Rivest, R. L., and Stein, C. (CLRS), Introduction to Algorithms — minimum spanning trees, disjoint sets, and dynamic programming.
- A standard algorithms textbook or peer-reviewed reference covering the multiple-knapsack problem and pseudo-polynomial dynamic programming.
- Official course assignment sheet for CT-363, cited according to the instructor's preferred format.

Do not include a reference in the final bibliography unless its bibliographic details have been checked and the source was actually consulted.

19. Final Submission Readiness Checklist

- The source code runs from a clean setup using the documented instructions.
- The submitted dataset contains at least 10 named network locations, at least 20 road connections, and at least 20 EV requests.
- The Kruskal implementation filters unavailable roads, applies the exact documented tie-break, returns an MSF when disconnected, and reports component names.
- The incremental three-road failure test repairs the existing forest and is checked against full Kruskal on the same modified graph.
- The DP recurrence, base case, capacity units, eligibility rule, tie-breaking, reconstruction, and complexity description match the implementation.
- The DP test scale is computationally feasible; station count and capacity discretization are disclosed.
- The three mandatory edge cases—disconnected graph, over-demand, and too-heavy request—are run and their actual outputs are included.
- The worked example and all sensitivity results are generated by the submitted code and are reproducible.
- The high-severity removal experiment removes the two highest-severity locations and all incident edges, then reruns both network and allocation stages.

- Every table and figure is labeled correctly; illustrative examples are not presented as measured results.
- The report includes correctness arguments, exact complexity derivations, limitations, and references actually consulted.
- The final report follows any additional formatting or submission instructions given by the instructor.